

UNAB

Programacion Avanzada

Guia de UML

Enfoque principal: Diagrama de Clases

Material de estudio	UML aplicado al diseno orientado a objetos
Tema central	Diagramas de clases: atributos, metodos, relaciones y multiplicidad
Incluye	Ejemplos de casos de uso, secuencia, actividad y objetos

Objetivo: que puedas leer, explicar y construir diagramas UML, especialmente diagramas de clases, con criterio de Programacion Avanzada.

Universidad Nacional Guillermo Brown - UNAB

Material academico de apoyo

Indice

- 1. Que es UML y para que sirve
- 2. UML dentro del analisis y diseno de software
- 3. Tipos de diagramas UML
- 4. Diagrama de clases: concepto principal
- 5. Notacion de clases: atributos, metodos y visibilidad
- 6. Relaciones en diagramas de clases
- 7. Multiplicidad
- 8. Ejemplo completo: sistema de biblioteca
- 9. Ejemplo completo: sistema de ventas
- 10. Otros diagramas UML: descripcion, ejemplo y ambito de uso
- 11. Comparacion rapida entre diagramas
- 12. Errores comunes
- 13. Guia paso a paso
- 14. Ejercicios y mini simulacro

1. Que es UML y para que sirve

UML significa **Unified Modeling Language**, o Lenguaje Unificado de Modelado. Es un lenguaje visual usado para representar sistemas de software mediante diagramas. No es un lenguaje de programacion: no se compila, no se ejecuta y no reemplaza al codigo. Su funcion es ayudar a pensar, comunicar y documentar un sistema antes o durante su desarrollo.

Una forma simple de verlo: **UML es el plano de una casa, pero aplicado a un sistema de software**. El codigo seria la construccion; UML ayuda a decidir como organizar esa construccion antes de levantar paredes a lo loco.

UML se usa para analizar requerimientos, disenar clases, comunicar decisiones tecnicas, explicar el flujo de un proceso y documentar sistemas. En Programacion Avanzada importa especialmente porque permite conectar el mundo conceptual de la Programacion Orientada a Objetos con una representacion clara y formal.

Idea clave: UML no sirve para dibujar por dibujar. Sirve para razonar mejor el software.

2. UML dentro del analisis y diseno de software

En un proyecto real, UML puede aparecer en distintas etapas. En analisis ayuda a entender que debe hacer el sistema. En diseno ayuda a decidir como se organizan clases, objetos y responsabilidades. En mantenimiento permite comprender sistemas existentes sin leer miles de lineas de codigo.

Etapas	Uso de UML	Ejemplo
Analisis	Entender actores, casos de uso y procesos.	Un socio solicita un prestamo de libro.
Diseno	Definir clases, relaciones, metodos y responsabilidades.	Prestamo se compone de DetallePrestamo.
Implementacion	Guiar la programacion de clases y servicios.	Crear clase Factura y clase DetalleFactura.
Documentacion	Explicar como funciona el sistema.	Diagrama de secuencia para una operacion critica.

3. Tipos de diagramas UML

UML tiene varios diagramas. Algunos muestran la estructura del sistema; otros muestran comportamiento o interacciones. En esta guia el centro es el **diagrama de clases**, porque es el mas importante para relacionar UML con POO y Programacion Avanzada.

Diagrama	Que representa	Cuando se usa
Clases	Estructura estatica: clases, atributos, metodos y relaciones.	Diseno orientado a objetos, arquitectura del dominio, parcial de POO.
Casos de uso	Actores y funcionalidades visibles para el usuario.	Analisis de requerimientos y alcance del sistema.
Secuencia	Mensajes entre objetos a traves del tiempo.	Explicar una operacion puntual: login, compra, prestamo.
Actividad	Flujo de pasos, decisiones y procesos.	Modelar procedimientos, algoritmos o circuitos administrativos.
Objetos	Instancias concretas en un momento determinado.	Ejemplificar el estado real del sistema.

Prioridad en esta guia: aproximadamente el 60% del contenido se concentra en diagramas de clases.

4. Diagrama de clases: concepto principal

El **diagrama de clases** representa la estructura estatica de un sistema. Muestra que clases existen, que datos guarda cada una, que operaciones puede realizar y como se relacionan entre si. Es estatico porque no muestra el paso del tiempo; muestra la arquitectura de objetos del sistema.

En Programacion Avanzada, este diagrama permite pasar de un enunciado a un diseno. Por ejemplo, si el enunciado dice que un cliente realiza muchas facturas, el diagrama debe reflejar la clase Cliente, la clase Factura y la relacion entre ambas con multiplicidad.

Un diagrama de clases bien hecho responde preguntas como: que clases necesito, que responsabilidad tiene cada una, que datos almacena, que metodos ofrece, quien conoce a quien y que objetos dependen fuertemente de otros.

Estructura basica de una clase UML

NombreClase
- atributo: Tipo
+ atributo2: Tipo
+ metodo(): Tipo
- metodoPrivado(): void

La caja se divide en tres partes: arriba va el nombre de la clase, en el medio los atributos y abajo los metodos. Si el diagrama es conceptual, a veces se omiten metodos o tipos; si es de diseno, conviene ser mas preciso.

5. Notacion de clases: atributos, metodos y visibilidad

La notacion clasica de un atributo es: **visibilidad nombre: Tipo**. La notacion clasica de un metodo es: **visibilidad nombreMetodo(parametros): TipoRetorno**.

Simbolo	Visibilidad	Significado	Ejemplo
+	Publico	Puede ser usado desde otras clases.	+ calcularTotal(): Float
-	Privado	Solo puede usarse dentro de la clase.	- saldo: Float
#	Protegido	Visible para la clase y sus hijas.	# validarEstado(): Boolean
~	Paquete/default	Visible dentro del mismo paquete/modulo.	~ generarCodigo(): String

En diseno orientado a objetos tradicional, los atributos suelen ser **privados** y los metodos que exponen comportamiento suelen ser **publicos**. Esto respeta encapsulamiento: los datos no se manipulan desde afuera como si el objeto fuera una bolsa abierta.

Ejemplos de atributos:

```
- id: Integer
- nombre: String
- precio: Float
- activo: Boolean
- fechaAlta: Date
```

Ejemplos de metodos:

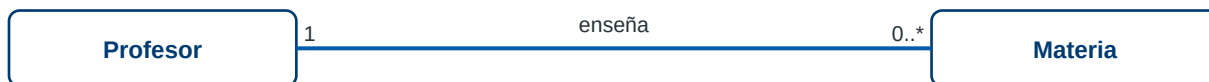
```
+ calcularTotal(): Float
+ iniciarSesion(email: String, password: String): Boolean
+ guardar(): void
+ marcarDisponible(): void
```

6. Relaciones en diagramas de clases

Las relaciones son lo mas importante del diagrama de clases. Una clase aislada dice poco; el diseno aparece cuando se entiende como colaboran las clases. Las relaciones principales son asociacion, herencia, composicion, agregacion y dependencia.

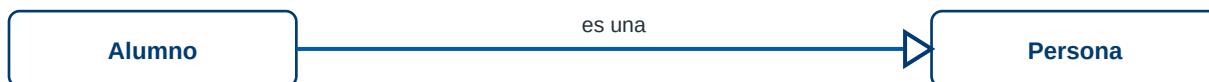
6.1 Asociacion

Una clase se vincula con otra. No implica propiedad fuerte ni ciclo de vida compartido. Ejemplo: Profesor ensena Materia.



6.2 Herencia / Generalizacion

Una clase hija especializa a una clase padre. Se lee como "es un/una". Ejemplo: Alumno es una Persona.



6.3 Composicion

Relacion fuerte de todo-parte. Si desaparece el todo, desaparecen las partes. Ejemplo: Factura contiene DetalleFactura.



6.4 Agregacion

Relacion debil de todo-parte. El elemento contenido puede existir aunque el agrupador desaparezca. Ejemplo: Curso agrupa Alumnos.



6.5 Dependencia

Una clase usa a otra de manera temporal o puntual. Ejemplo: ReporteService usa Impresora para imprimir.



7. Multiplicidad

La multiplicidad indica cuantos objetos de una clase pueden relacionarse con objetos de otra clase. Es clave porque muchas veces el error del diagrama no esta en las clases, sino en las cantidades.

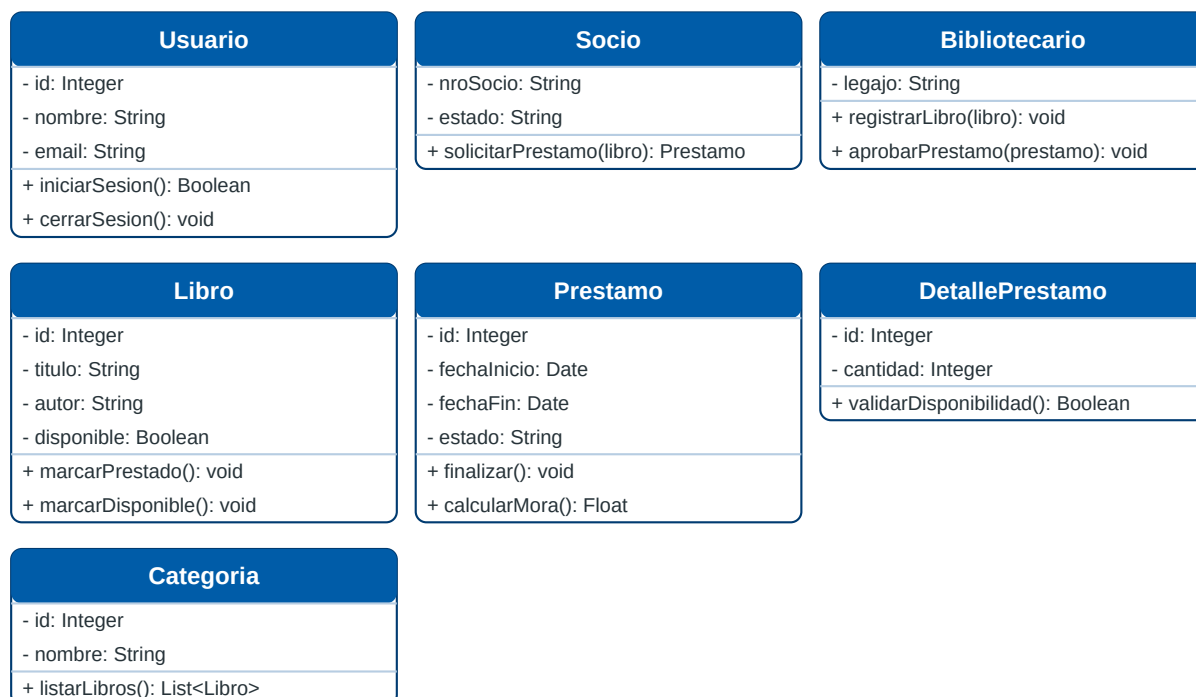
Multiplicidad	Significado	Ejemplo
1	Exactamente uno.	Una factura pertenece a un cliente.
0..1	Cero o uno.	Un usuario puede tener o no tener perfil adicional.
* / 0..*	Cero o muchos.	Un cliente puede tener muchas facturas o ninguna.
1..*	Uno o muchos.	Una factura debe tener al menos un detalle.
n..m	Entre n y m.	Un equipo puede tener entre 5 y 10 jugadores.

Lectura recomendada: leer la multiplicidad desde una clase hacia la otra. Ejemplo: Cliente 1 --- 0..* Factura significa: un cliente puede tener cero o muchas facturas, y cada factura pertenece a un cliente.

8. Ejemplo completo: sistema de biblioteca

Enunciado: una biblioteca tiene socios que solicitan prestamos de libros. Un bibliotecario registra libros y aprueba prestamos. Cada prestamo puede incluir uno o mas detalles, y cada detalle referencia un libro. Los libros pertenecen a una categoria.

Clases principales:



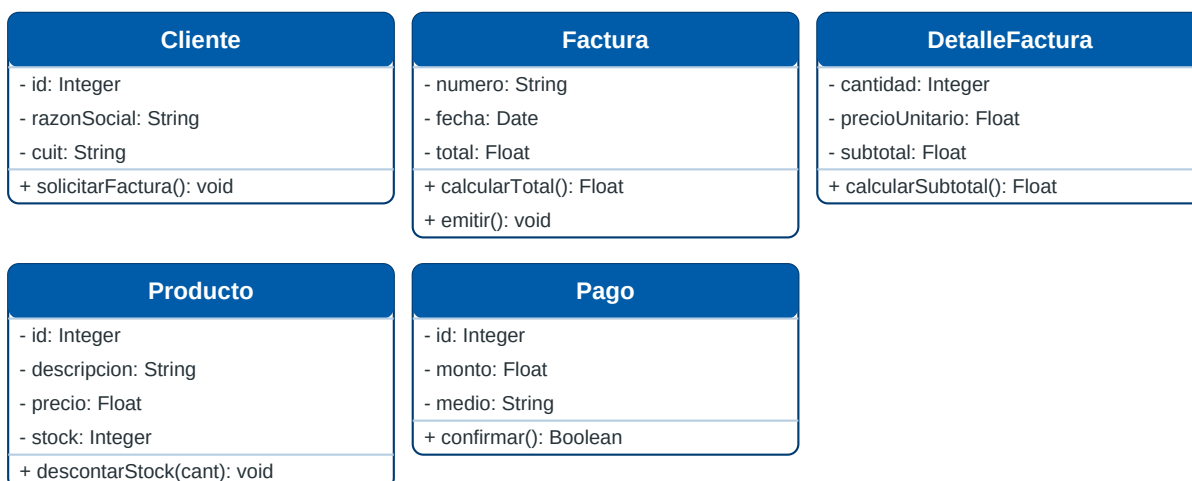
Relaciones del ejemplo:

Relacion	Tipo UML	Justificacion
Socio -> Usuario	Herencia	Socio es un tipo especializado de Usuario.
Bibliotecario -> Usuario	Herencia	Bibliotecario tambien es Usuario, pero con responsabilidades distintas.
Socio 1 -> 0..* Prestamo	Asociacion	Un socio puede tener varios prestamos; cada prestamo pertenece a un socio.
Prestamo 1 -> 1..* DetallePrestamo	Composicion	El detalle no tiene sentido sin el prestamo. Si se elimina el prestamo, se eliminan sus detalles.
DetallePrestamo * -> 1 Libro	Asociacion	El detalle referencia un libro, pero el libro existe por si mismo.
Categoria 1 -> 0..* Libro	Agregacion/Asociacion	La categoria agrupa libros, pero los libros podrian recategorizarse o existir en otra categoria.

Este ejemplo es tipico de parcial: tiene herencia, asociacion, composicion y multiplicidad. La parte mas importante es justificar por que Prestamo-DetallePrestamo es composicion y por que DetallePrestamo-Libro no lo es.

9. Ejemplo completo: sistema de ventas

Enunciado: un cliente puede realizar muchas facturas. Cada factura tiene uno o mas detalles. Cada detalle indica un producto vendido. Una factura puede pagarse con uno o varios pagos.



Relacion	Multiplicidad	Tipo recomendado	Motivo
Cliente - Factura	Cliente 1 / Factura 0..*	Asociacion	La factura pertenece a un cliente, pero el cliente no es una parte interna de la factura.
Factura - DetalleFactura	Factura 1 / Detalle 1..*	Composicion	El detalle existe dentro de la factura; sin factura no tiene sentido.
DetalleFactura - Producto	Detalle * / Producto 1	Asociacion	El producto existe en catalogo aunque se borre una factura.
Factura - Pago	Factura 1 / Pago 1..*	Composicion o asociacion	Si el pago queda como registro interno de la factura, composicion. Si pertenece a otro modulo contable, asociacion.

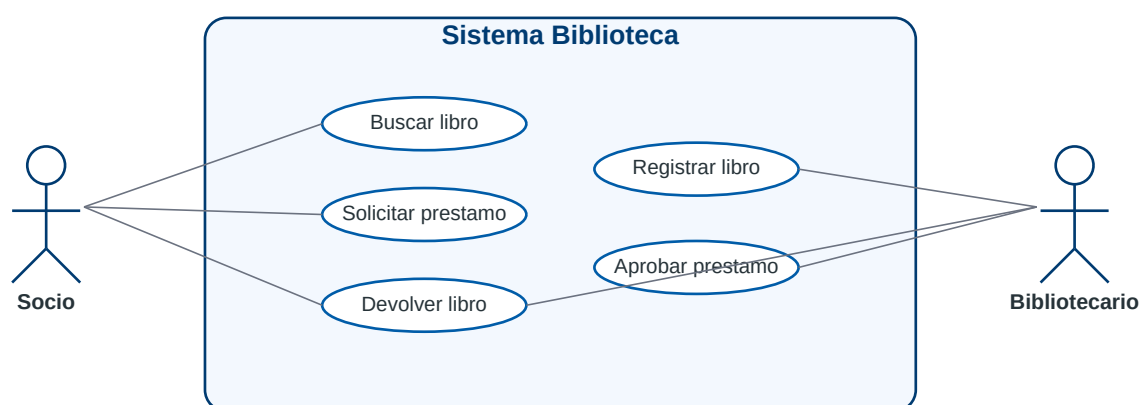
Regla practica: cuando una clase representa una linea interna de otra, como DetalleFactura, DetallePedido o ItemCarrito, casi siempre aparece composicion. El rombo negro va del lado del todo: Factura.

10. Otros diagramas UML: descripcion, ejemplo y ambito de uso

Aunque el foco de la materia suele caer fuerte en clases, otros diagramas ayudan a mirar el sistema desde otras perspectivas. No compiten con el diagrama de clases: lo complementan.

10.1 Diagrama de casos de uso

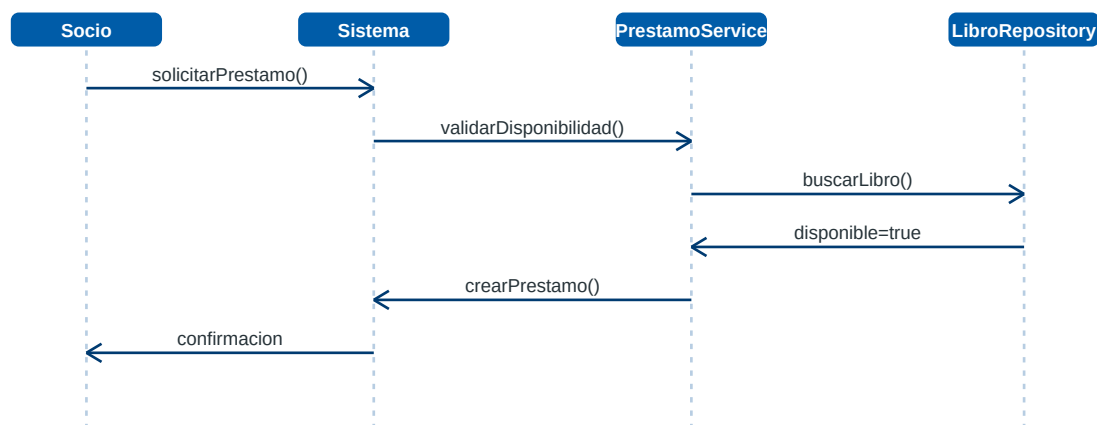
Describe que funcionalidades ofrece el sistema y que actores externos interactuan con el. Se usa en analisis de requerimientos, entrevistas con usuarios, alcance funcional y primeras etapas del proyecto. No muestra clases ni codigo: muestra objetivos del usuario.



Ámbitos de uso: relevamiento de requerimientos, documentacion inicial, validacion con usuarios no tecnicos y definicion de alcance.

10.2 Diagrama de secuencia

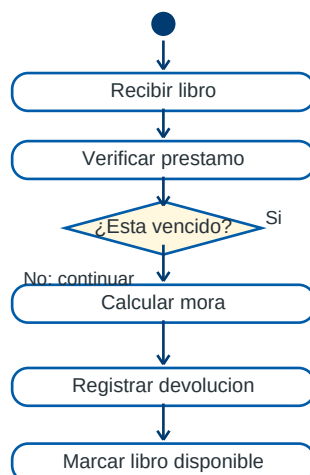
Muestra como interactuan objetos o componentes a lo largo del tiempo. Sirve para explicar una operacion concreta, por ejemplo solicitar un prestamo, iniciar sesion, confirmar una compra o emitir una factura.



Ámbitos de uso: diseno de servicios, APIs, controladores, casos donde importa el orden de llamadas y comunicacion entre objetos.

10.3 Diagrama de actividad

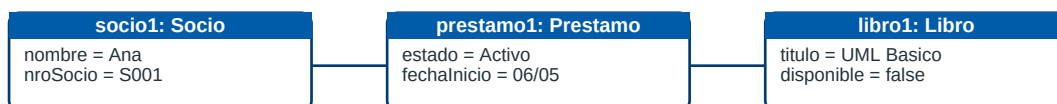
Representa un flujo de trabajo con pasos, decisiones y caminos alternativos. Se parece a un diagrama de flujo, pero con notación UML. Sirve para procesos administrativos, algoritmos, circuitos de aprobación y operaciones con varias decisiones.



Ambitos de uso: procesos de negocio, logica de validacion, flujos administrativos, algoritmos y procedimientos repetibles.

10.4 Diagrama de objetos

Muestra instancias concretas en un momento determinado. Mientras el diagrama de clases dice que existe la clase Socio, el diagrama de objetos puede mostrar socio1: Socio con nombre = Ana. Es una fotografia del sistema en ejecucion.



Ambitos de uso: explicar ejemplos, verificar relaciones con datos concretos, preparar casos de prueba y entender estados del sistema.

11. Comparacion rapida entre diagramas

Diagrama	Pregunta que responde	Principal ambito
Clases	¿Como esta estructurado el sistema?	Diseño orientado a objetos.
Casos de uso	¿Que puede hacer cada actor?	Requerimientos funcionales.
Secuencia	¿En que orden se comunican los objetos?	Operacion puntual o caso de uso interno.
Actividad	¿Que pasos sigue el proceso?	Flujos de trabajo y decisiones.
Objetos	¿Como se ve el sistema en un instante concreto?	Ejemplos, pruebas y estados.

12. Errores comunes en diagramas de clases

Error	Por que esta mal	Como corregirlo
Confundir clase con objeto	Cliente es clase; clienteJuan es objeto.	Usar nombres generales para clases.
Usar composicion para todo	No toda relacion es todo-parte fuerte.	Preguntar: si borro el todo, ¿la parte desaparece?
Olvidar multiplicidad	El diagrama queda ambiguo.	Agregar 1, 0..*, 1..*, etc.
Poner atributos publicos sin criterio	Rompe encapsulamiento.	Usar atributos privados y metodos publicos.
Meter frases largas como metodos	Un metodo debe ser una operacion clara.	Usar verbos concretos: calcularTotal(), finalizar().
Crear clases sin responsabilidad	El modelo se llena de ruido.	Cada clase debe representar una entidad o responsabilidad real.

Consejo conservador y efectivo: antes de ponerse creativo con flechas raras, dominar bien asociacion, herencia, composicion y multiplicidad. Lo tradicional funciona porque evita caos.

13. Guia paso a paso para construir un diagrama de clases

- Leer el enunciado completo sin dibujar todavia.
- Subrayar sustantivos importantes: Cliente, Factura, Producto, Prestamo.
- Convertir solo los sustantivos relevantes en clases.
- Detectar datos propios de cada clase para formar atributos.
- Detectar acciones o responsabilidades para formar metodos.
- Identificar relaciones entre clases.
- Decidir si cada relacion es asociacion, herencia, agregacion, composicion o dependencia.
- Agregar multiplicidades en ambos extremos.
- Revisar si hay clases repetidas, innecesarias o mal nombradas.
- Validar el diagrama contra el enunciado.

Ejemplo rapido:

Enunciado: "Un cliente puede realizar muchos pedidos. Cada pedido contiene uno o mas productos".

Elemento detectado	Decision UML
Cliente	Clase
Pedido	Clase
Producto	Clase
DetallePedido	Clase necesaria para representar cantidad, precio y subtotal
Cliente - Pedido	Asociacion 1 a 0..*
Pedido - DetallePedido	Composicion 1 a 1..*
DetallePedido - Producto	Asociacion * a 1

14. Ejercicios propuestos y mini simulacro

Ejercicios propuestos

Ejercicio	Consigna
Turnos medicos	Modelar Paciente, Medico, Turno y Especialidad. Indicar atributos, metodos, relaciones y multiplicidad.
Cursos	Modelar Alumno, Profesor, Curso e Inscripcion. Justificar si Inscripcion es clase necesaria.
Comercio	Modelar Cliente, Producto, Carrito, ItemCarrito, Pedido y Pago. Identificar composiciones.

Mini simulacro de parcial

- Definir UML y explicar por que no es un lenguaje de programacion.
- Explicar para que sirve un diagrama de clases.
- Diferenciar asociacion, agregacion y composicion con ejemplos.
- Dado un enunciado de ventas, identificar clases, atributos y metodos.
- Agregar multiplicidades correctas entre Cliente, Factura, DetalleFactura y Producto.
- Justificar por que Factura y DetalleFactura forman una composicion.

Solucion orientativa breve: UML modela visualmente sistemas; el diagrama de clases representa estructura estatica; asociacion vincula, agregacion agrupa debilmente y composicion contiene fuertemente; Factura-DetalleFactura es composicion porque el detalle depende de la factura para existir.

15. Cierre

El diagrama de clases es uno de los diagramas UML mas importantes para Programacion Avanzada porque obliga a pensar el sistema antes de programarlo. Permite ordenar clases, responsabilidades y relaciones. Cuando se domina bien, el codigo sale mas limpio, mas mantenible y menos improvisado.

La regla de oro: primero entender el dominio, despues dibujar clases, despues justificar relaciones. UML no premia el dibujo lindo si el razonamiento esta flojo. Mejor pocas clases bien justificadas que una pared llena de cajas sin sentido.